C PROGRAMMING ASSESSMENT 1

Name : Baranitha A (1965260049)

TASK 1 – Understand the Problem and Identify the Data

1. Inputs and Outputs

Inputs	Outputs
Previous reading	Units consumed
Current reading	Energy charge
Household number	Fixed charge
Category	Total bill

2. Constants

- First 100 units – Free.
- Next 100 units – Rs. 2.25/unit.
- Above 200 units – Rs. 4.50/unit.
- Fixed charge – Rs. 50.
- Maximum meter reading – 65535.

3. Data Values

- Negative values: Invalid readings or incorrect bills.
- Fractional values: Electricity charges in paise.
- Large values: Total bills for lakhs of households.

4. Billing Rules

- Bill should never be negative.
- Total = Fixed charge + Energy charge.
- Meter rollover must be handled correctly.
- Bills must be accurate to the nearest paisa.

5. Assumptions

- Each house has one meter.
- Meter uses a 16-bit unsigned counter.
- Tariff rates are the same for all households.

6. Alternative Concept – Integer vs String

Integer	String
Less memory	More memory

Faster comparison	Slower comparison
O(1) time	O(d) time

Here, d represents the number of digits.

Result: Integer representation is preferred because it requires less memory and provides faster calculations.

TASK 2 – Choose the Data Types

1. Data Type Selection

Data	Data type
Previous reading	unsigned short
Current reading	unsigned short
Units consumed	unsigned int
Household number	int
Bill amount	long
Total bill	long

2. Why use Integer Paise?

Float may produce rounding errors when adding many bills. Therefore, money is stored as integer paise.

Example:

- Rs. 50 = 5000 paise
- Rs. 2.25 = 225 paise
- Rs. 4.50 = 450 paise

3. Code :

```
#include <stdio.h>
int main()
{
   unsigned short a = 65400, b = 120;
   unsigned int units;
   units = (unsigned short)(b - a);
   printf("Consumption: %u units", units);
   return 0;
}
```

4. Output



5. Explanation

When the meter reaches 65535, it rolls over to 0. Therefore:

\[(65535-65400)+1+120=256 \]

Unsigned arithmetic prevents a negative result.

6. Alternative Concept – Double vs Integer Paise

- Double stores money in decimal form.
- Integer paise avoids rounding errors.
- Both take O(1) time for addition.
- Double usually occupies 8 bytes.
- Integer paise is more suitable for accurate billing.

Result: Integer paise and unsigned short are selected for accurate meter readings and billing

TASK 3 – Define the Constants

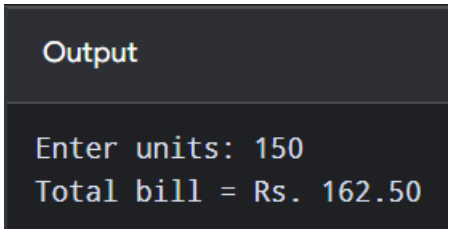
1. Constants

- First slab – 100 units.
- Second slab – 200 units.
- Rate 1 – 0 paise.
- Rate 2 – 225 paise.
- Rate 3 – 450 paise.
- Fixed charge – 5000 paise.

2. C Program

```
#include <stdio.h>#define RATE1 0
#define RATE2 225
#define RATE3 450
#define FIXED 5000
int main()
{
   int units;
   int bill = FIXED;
   printf("Enter units: ");
   scanf("%d", &units);
   if(units > 200)
      bill += (units - 200) * RATE3;
   if(units > 100)
   {
      if(units > 200)
         bill += 100 * RATE2;
      else
         bill += (units - 100) * RATE2;
   }
   printf("Total bill = Rs. %d.%02d", bill / 100, bill % 100);
   return 0;
}
```

3. Sample Output



4. Difference Between #define and const

#define	const
Preprocessor directive	Constant variable
No type checking	Type checking
No storage required	May use memory

5. Alternative Concept

- #define defines constants before compilation.
- const prevents a variable's value from changing.
- enum defines named integer constants.
- Running time is generally unaffected.

Result

Constants are defined using #define so that tariff changes can be made in one place.

TASK 4 – Use Macros Safely

1. Macros

Macros are used to perform repeated calculations easily. They are defined using #define.

2. C Program

```
#include <stdio.h>
#define SQUARE(x) ((x) * (x))
#define MAX(a,b) ((a) > (b) ? (a) : (b))
int main()
{
    int a = 5, b = 10;
    printf("Square = %d\n", SQUARE(a + 1));
    printf("Maximum = %d\n", MAX(a, b));
    return 0;
}
```

3. Output

```
Output

Square = 36
Maximum = 10
```

4. Macro Expansion

The command gcc -E program.c shows how macros are expanded before compilation.

5. Macro Problems

- Without parentheses, SQUARE(a+1) may give a wrong result.
- MAX(a,b) may evaluate an argument more than once.
- Functions are safer for complex calculations.

6. Alternative Concept – Inline Functions

- Inline functions work like normal functions.
- They provide type checking.
- They avoid repeated macro argument evaluation.
- Both can take O(1) time for simple calculations.
- Macros may increase code size when used many times because they are expanded at every use.

Result

Macros are defined with proper parentheses to avoid calculation errors.

TASK 5 – Organise the Program with Header Files and Conditional Compilation

1. Header File (billing.h)

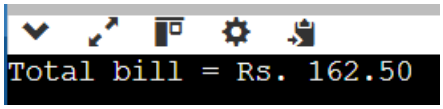
```
#ifndef BILLING_H
#define BILLING_H
#define FIXED 5000
long calculate_bill(int units);
#endif
```

2. C Program (main.c)

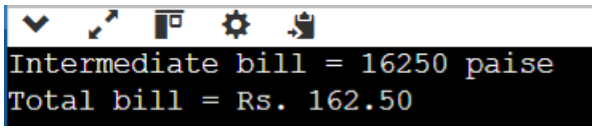
```c
#include <stdio.h>
#include "billing.h"
long calculate_bill(int units)
{
    long bill = FIXED;
    if(units > 100)
        bill += (units - 100) * 225;
    #ifdef DEBUG
    printf("Intermediate bill = %ld paise\n", bill);
    #endif
    return bill;
}
int main()
{
    int units = 150;
    long bill = calculate_bill(units);
    printf("Total bill = Rs. %ld.%02ld\n", bill / 100, bill % 100);
    return 0;
}
```

3. Output

Normal compilation:



Debug compilation:



4. Compilation Commands

gcc main.c -o bill

./bill

For debug mode:

gcc -DDEBUG main.c -o bill

./bill

5. Explanation

- Header files contain constants and function declarations.
- Include guards prevent a header file from being included multiple times.
- #ifdef DEBUG displays intermediate values only in debug mode.
- #include <file> is used for standard header files.
- #include "file" is used for user-created header files.

6. Alternative Concept

- Run-time debugging uses if(debug) to check whether debugging is enabled.
- Conditional compilation uses #ifdef DEBUG.
- Run-time checking takes an extra O(1) test.
- Conditional compilation removes debug code when DEBUG is not defined.

Result

Header files and conditional compilation are used to organise the program and simplify debugging.

Task 6 – scanf() and printf() Format Specifiers

1. Format specifiers

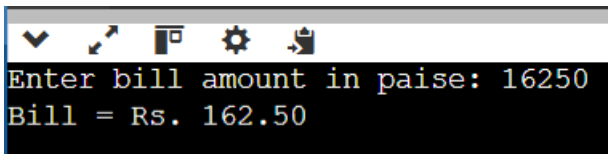
Data type	Format specifier
int	%d
unsigned int	%u
long	%ld
float	%f
double	%lf in scanf
char	%c
string	%s

2. Program

```c
#include <stdio.h>

int main()
{
    long paise;
    printf("Enter bill amount in paise: ");
    if(scanf("%ld", &paise) != 1)
    {
        printf("Invalid input");
        return 1;
    }
    printf("Bill = Rs. %ld.%02ld\n", paise / 100, paise % 100);
    return 0;
}
```

3. Output



4. Explanation

- %ld is used to read a long integer.
- scanf() checks whether the input is valid.
- If the input is invalid, it displays an error message.
- Division by 100 gives rupees, and remainder gives paise.
- %02ld displays paise using two digits.

5. Alternative method

fgets() and strtol() can be used to read and validate input safely. They help handle invalid input more effectively.

6. Complexity

- Time complexity: O(L), where L is the input length.
- Space complexity: O(L) for the input buffer.

Task 7 – Type Conversion and Precision

1. Type conversion

Type conversion means changing a value from one data type to another. It can happen automatically or by using type casting.

2. Integer division

When two integers are divided, the decimal part is removed.

Example: 5 / 2 = 2

To get the decimal value, use type casting: (float)5 / 2 = 2.50.

3. C Program

```
#include <stdio.h>

int main()

{

   int a = 5, b = 2;

   float x = 10 / 3;

   double y = 0.1 + 0.2;

   printf("Integer division = %d\n", a / b);

   printf("Type casting = %.2f\n", (float)a / b);

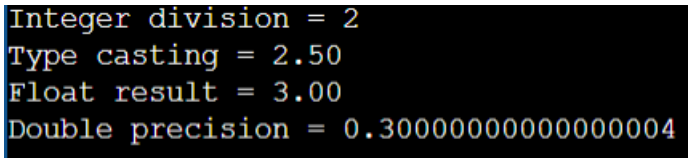
   printf("Float result = %.2f\n", x);

   printf("Double precision = %.17f\n", y);

   return 0;

}
```

4. Output



5. Explanation

- Integer division removes the decimal part.
- Type casting helps obtain decimal results.
- 10 / 3 gives 3 because both values are integers.
- Floating-point values may have small precision errors.

6. Rounding

The round() function can be used to round a value to the nearest integer. For displaying money, %.2f displays two decimal places.

7. Alternative

Using double provides better precision than float. For electricity billing, storing money as integer paise is more accurate and avoids floating-point errors.

Conclusion: Proper type conversion and precision handling help prevent calculation errors in billing systems.

Task 8 – Compilation Stages in C

1. Compilation process

A C program passes through four main stages:

1. Preprocessing: Processes header files and macros.
2. Compilation: Converts C code into assembly code.
3. Assembly: Converts assembly code into object code.
4. Linking: Combines object code and libraries to create an executable file.

2. GCC commands

gcc -E main.c -o main.i

gcc -S main.c -o main.s

gcc -c main.c -o main.o

gcc main.o -o main

3. Explanation

- -E performs preprocessing.
- -S generates assembly code.
- -c generates object code.
- The final command links the object file and creates an executable.

4. Errors and warnings

- Syntax errors: Mistakes in program structure.
- Warnings: Possible problems in the code.
- Linker errors: Occur when required functions or libraries are missing.

5. Multi-file compilation

Multiple C files can be compiled together:

gcc main.c billing.c -o billing

Conclusion: Compilation converts C source code into an executable program through four stages.

Task 9 – Complete Electricity Billing Program

1. Objective

To develop a C program that calculates electricity bills based on the number of units consumed.

2. Tariff details

- Fixed charge = Rs. 50
- First 100 units = Free
- Next 100 units = Rs. 2.25 per unit
- Above 200 units = Rs. 4.50 per unit

3. Program

```
#include <stdio.h>
int main()
{
   int units;
   float bill = 50;
   printf("Enter units: ");
   scanf("%d", &units);
   if(units > 100)
      bill += (units - 100) * 2.25;
   if(units > 200)
      bill += (units - 200) * 2.25;
   printf("Total bill = Rs. %.2f\n", bill);
   return 0;
}
```

4. Sample output

```
Enter units: 150
Total bill = Rs. 162.50
```

```
Enter units: 350
Total bill = Rs. 950.00
```

5. Testing

Units	Bill
0	Rs. 50.00
100	Rs. 50.00
150	Rs. 162.50
200	Rs. 275.00
350	Rs. 950.00

6. Conclusion

The program successfully calculates electricity bills using tariff rates and fixed charges. It was tested with different unit values.

Task 10 – Accuracy and Overflow Analysis

1. Objective

To compare floating-point calculations with integer paise calculations and identify possible errors.

2. C Program

```
#include <stdio.h>
int main()
{
    int units = 4000;
    long bill;
    bill = 5000 + (units - 100) * 225;
    printf("Total bill = Rs. %ld.%02ld\n", bill / 100, bill % 100);
    return 0;
}
```

Output :

```
Total bill = Rs. 8825.00
```

3. Comparison

Total units	Billing amount
4,000	Rs. 17,375
60,000	Rs. 269,375
800,000	Rs. 3,599,375

4. Explanation

- Floating-point values may produce small rounding errors.
- Integer paise calculations provide accurate money values.
- Small integer data types may cause overflow for large readings.
- double provides better precision than float.
- long long can store larger integer values.

5. Recommendation

Use integer paise for billing calculations and suitable large integer data types to avoid overflow.

6. Conclusion

Integer-based calculations improve accuracy and reliability in large-scale electricity billing systems.

Task 11 – Final Billing System Design

1. Objective

To recommend a reliable design for the rural electricity billing system.

2. Recommended design

- Use unsigned short for meter readings.
- Use long or long long for billing calculations.
- Store money in integer paise.
- Use macros for tariff rates and fixed charges.
- Validate all user inputs.
- Handle meter rollover correctly.

3. Tariff revision

Tariff rates should be stored separately so they can be changed easily without modifying the main program.

4. C Program

```
#include <stdio.h>
#define FIXED 5000
#define RATE 225
int main()
{
    int units = 150;
    long bill;
    bill = FIXED + (units - 100) * RATE;
    printf("Units = %d\n", units);
    printf("Bill = Rs. %ld.%02ld\n", bill / 100, bill % 100);
    return 0;
}
```

```
Units = 150
Bill = Rs. 162.50
```

5. Limitations

- Small data types may cause overflow.
- Invalid inputs can affect calculations.
- Hardcoded tariff values require recompilation when changed.

6. Alternative approach

A configuration file can store tariff rates and fixed charges, allowing updates without recompiling the program.

7. Conclusion

The recommended design provides accurate, reliable and maintainable electricity billing for the district.

Task 12 – Smart Meter Design Challenge

1. Objective

To develop a smart meter program that records electricity consumption and calculates bills based on peak-hour usage.

2. C Program

```
#include <stdio.h>

int main()

{

   int units = 150;

   int peak_units = 50;

   float bill;

   bill = (units - 100) * 2.25 + 50;

   bill += peak_units * 2.25 * 0.20;

   printf("Total units = %d\n", units);

   printf("Peak units = %d\n", peak_units);

   printf("Total bill = Rs. %.2f\n", bill);

   return 0;

}
```

3. Expected Output

```
Total units = 150
Peak units = 50
Total bill = Rs. 185.00
```

4. Explanation

- The meter records electricity consumption every 15 minutes.
- There are 96 readings per day.
- There are 35,040 readings per year.
- Peak hours are from 6 PM to 10 PM.
- An additional 20% charge is applied during peak hours.
- Tariff values can be stored separately for easy updates.

5. Advantages

- Accurate electricity monitoring.
- Automatic bill calculation.
- Supports peak-hour pricing.
- Helps reduce electricity wastage.

6. Conclusion

The smart meter system helps monitor electricity consumption and calculate bills efficiently using time-based tariffs.